

OE-EC803A — Internet of Things

Semester VIII • 3L:0T:0P • 3 Credits

Module 2: Internet Principles & Networking in IoT

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Internet Communications Overview	2
1.1	Transport Layer: TCP vs. UDP	2
2	IP Addressing and Routing	2
2.1	IP Addresses and DNS	3
2.2	IPv4 vs. IPv6 in IoT	3
2.3	MAC Addresses	3
2.4	Ports	3
3	Application Layer Protocols in IoT	3
3.1	MQTT (Message Queuing Telemetry Transport)	4
3.2	CoAP (Constrained Application Protocol)	4

1. Internet Communications Overview

IoT devices rely on internet communication stacks to transmit sensor data and receive commands. The dominant model is the **TCP/IP Protocol Suite**.

TCP/IP Protocol Suite

The **TCP/IP Protocol Suite** is a four-layer conceptual model for network communication. It consists of the Link Layer, Internet Layer (IP), Transport Layer (TCP/UDP), and Application Layer (HTTP/CoAP/MQTT).

1.1 Transport Layer: TCP vs. UDP

IoT edge devices choose transport protocols based on power, latency, and reliability requirements:

- **TCP (Transmission Control Protocol):** A connection-oriented, reliable protocol. It guarantees in-order delivery of data using handshakes, acknowledgments, and retransmission mechanisms. The drawback is high header overhead (20 bytes) and higher power consumption, which is challenging for constrained devices.
- **UDP (User Datagram Protocol):** A connectionless, lightweight transport protocol. It sends packets (datagrams) without verifying if the receiver is ready or if the packets arrive. It has minimal header overhead (8 bytes) and low latency, making it ideal for battery-powered sensor nodes (often combined with CoAP).

TCP Three-Way Handshake Connection Lifecycle

Establishing a reliable TCP connection requires a three-way handshake before any application data is sent:

1. **SYN (Synchronize):** The client sends a segment with the SYN flag set and an initial sequence number ($Seq = x$) to request a connection.
2. **SYN-ACK (Synchronize-Acknowledge):** The server replies with a segment where both SYN and ACK flags are set, acknowledging the client's sequence ($Ack = x + 1$) and sending its own sequence number ($Seq = y$).
3. **ACK (Acknowledge):** The client sends a final segment acknowledging the server's sequence ($Ack = y + 1$). The connection is now **ESTABLISHED**.

For teardown, a four-step handshake using **FIN** (Finish) and **ACK** segments is executed to close both directions of the full-duplex link.

2. IP Addressing and Routing

Every node connected to the Internet needs a unique address for routing packets.

2.1 IP Addresses and DNS

- **DNS (Domain Name System):** Translates human-readable domain names (e.g. 'api.weather.com') into machine-readable IP addresses.
- **Static IP Assignment:** The address is manually configured on the device. Ideal for servers and local gateways but hard to scale.
- **Dynamic IP Assignment (DHCP):** A DHCP server automatically assigns a temporary IP address to devices when they connect to the network. Essential for scaling large fleets of consumer IoT devices.

2.2 IPv4 vs. IPv6 in IoT

IPv6 Importance in IoT

IPv6 (Internet Protocol version 6) uses 128-bit addresses, yielding 3.4×10^{38} unique addresses, compared to the 32-bit (4.3 billion) address pool of **IPv4**. IPv6 is critical for IoT because it allows every smart object on Earth to have a unique, globally routable public IP address, eliminating the need for complex Network Address Translation (NAT) gateways.

Feature	IPv4 Header	IPv6 Header
Header Size	Variable (20 to 60 bytes).	Fixed (40 bytes).
Address Size	32 bits (4 bytes).	128 bits (16 bytes).
Fields Count	12 fields.	Only 8 fields.
Checksum Field	Present (must be recomputed at each hop).	Removed (handled by Layer 2 and Layer 4).
Fragmentation	Handled by routers and sending hosts.	Handled only by sending hosts (PMTU discovery).

2.3 MAC Addresses

A **MAC (Media Access Control) Address** is a unique, 48-bit physical hardware identifier burned into a network interface card (NIC) by the manufacturer. While IP addresses change based on location/network, the MAC address remains constant, uniquely identifying the hardware locally.

2.4 Ports

Ports are 16-bit logical identifiers at the Transport Layer used to route data packets to specific software applications running on a host. Common IoT port assignments include HTTP (80), MQTT (1883), CoAP (5683), and HTTPS/MQTTs (443/8883).

3. Application Layer Protocols in IoT

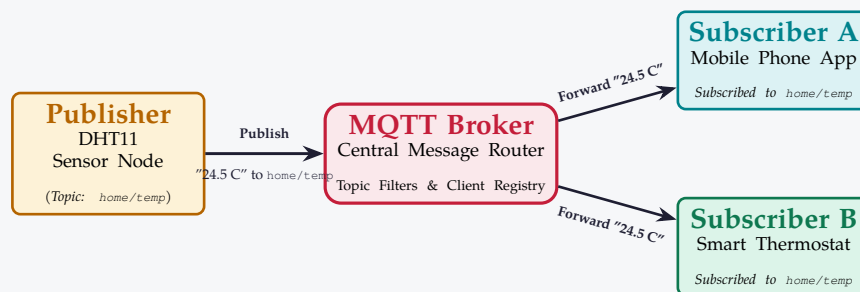
To handle resource constraints (memory, bandwidth, power), specialized application layer protocols are used instead of heavy web HTTP.

3.1 MQTT (Message Queuing Telemetry Transport)

MQTT is a lightweight, broker-based publish-subscribe messaging protocol designed for high-latency, low-bandwidth networks.

- **Clients:** Publish messages to topics or subscribe to topics to receive messages.
- **Broker:** The central server that receives published messages and distributes them to all subscribed clients.
- **QoS (Quality of Service) Levels:**
 - *QoS 0 (At most once):* Fire-and-forget; no handshake, no guaranteed delivery. Minimal overhead.
 - *QoS 1 (At least once):* Guaranteed delivery but duplicates may arrive.
 - Client sends a `PUBLISH` message containing a unique Packet Identifier.
 - Broker stores and forwards the message, replying with a `PUBACK` acknowledgment. If the client doesn't receive `PUBACK` before a timeout, it re-transmits.
 - *QoS 2 (Exactly once):* Four-step handshake guarantees delivery with zero duplicates.
 - Client sends `PUBLISH`.
 - Broker stores packet ID and sends `PUBREC` (Publish Received).
 - Client receives `PUBREC` and sends `PUBREL` (Publish Release) to unlock broker storage.
 - Broker processes/distributes message, replying with `PUBCOMP` (Publish Complete).

MQTT Publish-Subscribe Architecture



3.2 CoAP (Constrained Application Protocol)

CoAP is a web transfer protocol designed for constrained nodes and networks:

- It uses a RESTful request-response model mapping to standard HTTP methods (GET, POST, PUT, DELETE).
- Unlike HTTP, CoAP runs over **UDP** instead of TCP, dramatically reducing connection handshakes and packet overhead.

CoAP Request-Response over UDP

Sends stateless requests via UDP

Responds statelessly via UDP

Quick Revision — Module 2

Networking & Protocols | OE-EC803A

Key Protocols and Specs at a Glance

IPv4 Address Length	32 bits (4 bytes) → e.g., 192.168.1.10
IPv6 Address Length	128 bits (16 bytes) → e.g., 2001 : 0db8 : 85a3 :: 8a2e : 0370 : 7334
MQTT Ports	Port 1883 (TCP, unencrypted) and Port 8883 (Encrypted SSL/TLS)
CoAP Port	Port 5683 (UDP, unencrypted) and Port 5684 (DTLS security)

One-Line Definitions

- **TCP/IP Model:** A standardized suite of communication protocols used to connect devices on the internet.
- **QoS (MQTT):** Quality of Service agreement determining the reliability and handshake level of message delivery.
- **MAC Address:** A permanent, unique physical address burned into the hardware interface card.
- **CoAP:** A lightweight RESTful transfer protocol running over UDP for constrained networks.
- **DHCP:** An automated protocol that dynamically assigns IP configurations to joining devices.

Frequently Asked / Viva Questions

Q1. Why is TCP preferred for applications like web browsing, but UDP is preferred for real-time sensor streams?

TCP is connection-oriented and guarantees that every packet is received in order through acknowledgments. This prevents page layout distortion but adds latency. UDP sends packets without handshakes. It is faster, has less overhead, and is ideal for sensor streams where losing an occasional packet is acceptable but latency must be low.

Q2. Why is IPv6 essential for the growth of IoT?

IPv4 only supports 4.3 billion addresses, which have already been exhausted. IoT connects tens of billions of smart objects. IPv6 provides 128-bit addresses (3.4×10^{38}), allowing every individual device to have a globally unique IP address without routing behind NAT gateways.

Frequently Asked / Viva Questions (contd.)**Q3. How does the publish-subscribe model in MQTT differ from the client-server model in HTTP?**

In HTTP (client-server), clients pull data by initiating requests, requiring both devices to be online and active at the same time (tight temporal coupling). In MQTT (publish-subscribe), clients publish data to a broker, which pushes it to subscribers. The publisher and subscriber do not know each other and are decoupled.

Q4. Explain the three QoS levels in MQTT.

(1) QoS 0 (At most once): Sends the message once with zero handshakes. Best for high-frequency, non-critical data. (2) QoS 1 (At least once): Delivers the message but might send duplicates. Requires a puback packet. (3) QoS 2 (Exactly once): Guarantees the message is received exactly once using a 4-packet exchange.

Q5. What is CoAP, and what transport layer protocol does it use?

CoAP (Constrained Application Protocol) is a lightweight web transfer protocol designed for resource-constrained IoT nodes. It mirrors the RESTful model of HTTP (GET/POST) but runs over UDP, reducing header overhead and handshake latency.

Q6. Why do we need a DNS server?

Humans read names (like 'io.adafruit.com'), but routing hardware only reads binary IP addresses. The DNS server acts as a directory that looks up the domain name and returns the active IP address.

Q7. Explain the difference between static and dynamic IP assignment.

Static IP addresses are configured manually and remain fixed, which is useful for local gateways. Dynamic IP addresses are leased temporarily from a pool by a DHCP server, which simplifies administration for large scale deployments.

Q8. What is the difference between a MAC address and an IP address?

A MAC address is a physical, 48-bit identifier assigned permanently to the device hardware by the manufacturer. An IP address is a logical address assigned dynamically by the network to route packets across subnets.

Comparison Snapshot**MQTT vs. CoAP vs. HTTP:**

Feature	MQTT	CoAP	HTTP
Architecture	Publish / Subscribe	Request / Response (REST)	Request / Response (REST)
Transport Layer	TCP	UDP	TCP
Header Size	Minimal (2 bytes)	Small (4 bytes)	Large (Hundreds of bytes)
Security	SSL / TLS	DTLS	SSL / TLS (HTTPS)
Use Case	Continuous telemetry streams.	Low-power, event-driven queries.	Heavy dashboard interfaces.